

多线程

获取当前线程和当前活跃线程

```
1 import threading
2
3 #获取当前线程
4 t1 = threading.current_thread()
5 print(t1,id(t1))
6
7 t2 = threading.current_thread()
8 print(t2,id(t2))
9
10 #获取当前活跃线程
11 print(threading.active_count())
12
```

创建子线程

```
1 import threading
2
3 def thread_body():
4     t = threading.current_thread()
5     while True:
6         print(f'{t.name} 线程正在运行中.....')
7
8
9 if __name__ == '__main__':
10     t1 = threading.Thread(target=thread_body)
11     #t2 = threading.Thread(target=thread_body)
12     t1.start()
13     #t2.start()
14     while True:
15         print(f'{threading.current_thread().name}线程正在执行中.....')
```

子线程传参

```
1 import threading
2
3 def thread_body(movie_name):
4     t = threading.current_thread()
5     while True:
6         print(f'{t.name} 线程正在下载{movie_name}中.....')
7
8
9 if __name__ == '__main__':
10     #注意，一个参数的情况下要以逗号结尾，否则args接收到的不是元组，而是字符串
11     t1 = threading.Thread(target=thread_body, args=("哈利波特",))
12     t1.start()
13     while True:
14         print(f'{threading.current_thread().name}线程正在执行中.....')
```

daemon关键字

在预设情况下，所有的线程皆不是Daemon线程（可以翻译为守护线程），在默认情况下，如果一个程序建立了主线程于其它子线程，在所有线程工作结束后，程序才会结束.因为如果主线程若是先结束，将退回所有占据的资源给操作系统，如果子线程仍在执行将会因为没有资源造成程序崩溃.

但是当我们设计的线程是Daemon线程时，主线程若是想要结束执行，会检查Daemon 线程的属性.

1. 如果此时Daemon线程的属性是True，即使Daemon 线程仍在执行，其他非Daemon 线程执行结束，程序将不等待Daemon线程，也会自行结束同时终止此 Daemon 线程工作.
2. 如果此时Daemon线程的属性是False，主线程会等待Daemon线程结束，再结束工作.

```

1  import threading
2
3  def thread_body(movie_name):
4      t = threading.current_thread()
5      while True:
6          print(f'{t.daemon},{t.name} 线程正在下载{movie_name}中....')
7
8
9  if __name__ == '__main__':
10     #注意，一个参数的情况下要以逗号结尾，否则args接收到的不是元组，而是字符串
11     #daemon=True 主线程如果退出,则daemon线程也退出
12     t1 = threading.Thread(target=thread_body,args=("哈利波特",),name="下载任务",daemon=True)
13     t1.start()
14     print("主线程运行结束....")

```

join等待线程执行

```

1  import threading
2  import time
3
4
5  def thread_body(movie_name):
6      t = threading.current_thread()
7      for i in range(5):
8          print(f'{t.daemon},{t.name} 线程正在下载{movie_name}中....')
9          time.sleep(1)
10
11
12  if __name__ == '__main__':
13     t1 = threading.Thread(target=thread_body,args=("哈利波特",),name="下载任务")
14     t1.start()
15     #阻塞当前线程，等待子线程执行完毕
16     t1.join()
17     print("主线程运行结束....")

```

线程停止

```
1 import time
2 import threading
3
4 isrunning = True
5
6 def working_body():
7     while isrunning:
8         print("工作体线程正在执行中.....")
9         time.sleep(1)
10    print("工作体线程执行完毕")
11
12 def control_body():
13     #要修改变量，需要global修饰
14     global isrunning
15     while isrunning:
16         command = input() #接收用户输入指令
17         if command == 'exit':
18             isrunning = False
19         else:
20             print(command)
21
22
23 if __name__ == '__main__':
24     working_thread=threading.Thread(target=working_body,name="工作线程")
25     working_thread.start()
26
27     control_body = threading.Thread(target=control_body,name='控制线程')
28     control_body.start()
29
30    print("主线程执行完毕...")
31
```

自定义线程体实现多线程

```
1 import threading
2 import time
3
4 class DownloadThread(threading.Thread):
5
6     def __init__(self, movieName, threadName):
7         #必须调用父类初始化
8         super(DownloadThread, self).__init__(name=threadName)
9         self.movieName = movieName
10
11     #子线程类要重写父类的run方法，调用子线程的start方法会让run方然在独立的子线程中运行
12     #不能直接调用run方法
13     #start方法只能被调用一次
14     def run(self):
15         for i in range(10):
16             print(f'{threading.current_thread().name} 正在下载电影
17 {self.movieName}....')
18             time.sleep(1)
19
20 if __name__ == '__main__':
21     download_thread=DownloadThread(movieName="哈利波特", threadName="电影下载线程")
22     download_thread.start()
23     print("主线程运行完毕....")
```

多线程模拟银行取现(错误示范)

```
1 import threading
2 import time
3 import random
4
5 class Bank(object):
6     def __init__(self, money):
7         self.money = money
8
9     def get_money(self, number):
10        print(f'{threading.current_thread().name}正在执行取现操作....')
11        if number < 0 :
12            return -1
13        elif number > self.money:
14            return -2
15        elif self.money < 0:
16            return -3
17        else:
18            #模拟CPU切换,重现问题
19            time.sleep(random.random())
20            self.money -= number
21            print(f'left money : {self.money}')
22            return number
23
24
25 class FetchMoney(threading.Thread):
26
27     def __init__(self, bank, threadName):
28         super(FetchMoney, self).__init__(name=threadName)
29         self.bank = bank
30
31     def run(self):
32         number = self.bank.get_money(800)
33         print(f'取款成功{number}')
34
35
36 if __name__ == '__main__':
37     bank = Bank(1000)
```

```
38
39     t1=FetchMoney(bank,"柜台线程")
40     t2=FetchMoney(bank,"ATM线程")
41
42     t1.start()
43     t2.start()
44
45
```

多线程模拟银行取现(问题修复)

```
1 import threading
2 import time
3 import random
4
5 class Bank(object):
6     def __init__(self, money):
7         self.money = money
8         self.lock = threading.RLock()
9
10    def get_money(self, number):
11        print(f'{threading.current_thread().name}正在执行取现操作....')
12        self.lock.acquire()
13        if number < 0 :
14            self.lock.release()
15            return -1
16        elif number > self.money:
17            self.lock.release()
18            return -2
19        elif self.money < 0:
20            self.lock.release()
21            return -3
22        else:
23            #模拟CPU切换,重现问题
24            time.sleep(random.random())
25            self.money -= number
26            print(f'left money : {self.money}')
27            self.lock.release()
28            return number
29
30
31
32 class FetchMoney(threading.Thread):
33
34    def __init__(self, bank, threadName):
35        super(FetchMoney, self).__init__(name=threadName)
36        self.bank = bank
37
38    def run(self):
```

```
39         number = self.bank.get_money(800)
40         if(number > 0):
41             print(f'取款成功{number}')
42         else:
43             print(f'取款失败{number}')
44
45
46 if __name__ == '__main__':
47     bank = Bank(1000)
48
49     t1=FetchMoney(bank, "柜台线程")
50     t2=FetchMoney(bank, "ATM线程")
51
52     t1.start()
53     t2.start()
54
55
56
57
```

多线程模拟银行取现(优化)

```
1 import threading
2 import time
3 import random
4
5 class Bank(object):
6     def __init__(self, money):
7         self.money = money
8         self.lock = threading.RLock()
9
10    def get_money(self, number):
11        print(f'{threading.current_thread().name}正在执行取现操作....')
12        try:
13            self.lock.acquire()
14            if number < 0 :
15                self.lock.release()
16            elif number > self.money:
17                return -2
18            elif self.money < 0:
19                return -3
20            else:
21                #模拟CPU切换,重现问题
22                time.sleep(random.random())
23                self.money -= number
24                print(f'left money : {self.money}')
25                return number
26        finally:
27            self.lock.release()
28
29
30
31 class FetchMoney(threading.Thread):
32
33    def __init__(self, bank, threadName):
34        super(FetchMoney, self).__init__(name=threadName)
35        self.bank = bank
36
37    def run(self):
38        number = self.bank.get_money(800)
```

```
39         if(number > 0):
40             print(f'取款成功{number}')
41         else:
42             print(f'取款失败{number}')
43
44
45 if __name__ == '__main__':
46     bank = Bank(1000)
47
48     t1=FetchMoney(bank,"柜台线程")
49     t2=FetchMoney(bank,"ATM线程")
50
51     t1.start()
52     t2.start()
53
```

售票系统(错误演示)

```
1 import threading
2 import time
3 import random
4
5 class TicketWindow(threading.Thread):
6
7     tickets = 100
8
9     def __init__(self, threadName):
10         super(TicketWindow, self).__init__(name=threadName)
11
12
13     def run(self):
14         while True:
15             if TicketWindow.tickets > 0 :
16                 ticketIndex = TicketWindow.tickets
17                 print(f'{self.name} 正在售出第{ticketIndex}张票')
18                 time.sleep(random.random())
19                 TicketWindow.tickets -= 1
20                 print(f'{self.name} 已经售出第{ticketIndex}张票')
21             else:
22                 break
23
24
25 if __name__ == '__main__':
26     t1 = TicketWindow('窗口1')
27     t2 = TicketWindow('窗口2')
28     t3 = TicketWindow('窗口3')
29     t1.start()
30     t2.start()
31     t3.start()
32     t1.join()
33     t2.join()
34     t3.join()
35
36
```

售票系统(问题修复)

```
1 import threading
2 import time
3 import random
4
5 class TicketWindow(threading.Thread):
6
7     tickets = 100
8     lock = threading.RLock()
9
10    def __init__(self, threadName):
11        super(TicketWindow, self).__init__(name=threadName)
12
13
14    def run(self):
15        while True:
16            try:
17                TicketWindow.lock.acquire()
18                if TicketWindow.tickets > 0 :
19                    ticketIndex = TicketWindow.tickets
20                    print(f'{self.name} 正在售出第{ticketIndex}张票')
21                    time.sleep(random.random())
22                    TicketWindow.tickets -= 1
23                    print(f'{self.name} 已经售出第{ticketIndex}张票')
24                else:
25                    break
26            finally:
27                TicketWindow.lock.release()
28
29
30 if __name__ == '__main__':
31     t1 = TicketWindow('窗口1')
32     t2 = TicketWindow('窗口2')
33     t3 = TicketWindow('窗口3')
34     t1.start()
35     t2.start()
36     t3.start()
37     t1.join()
38     t2.join()
```

```
39     t3.join()
```

```
40
```

```
41
```

条件锁定

```
1 import random
2 import threading
3
4 def producer():
5     while True:
6         print(f'{threading.current_thread().name}:准备获取锁 ...')
7         condition.acquire()
8         print(f'{threading.current_thread().name}:已经获取锁 ...')
9         if len(data) >= 5:
10             print(f'{threading.current_thread().name}:waiting ...')
11             condition.wait()
12             print(f'{threading.current_thread().name}:wait 被唤醒')
13         else:
14             data.append(random.randint(1,100))
15             print(f'生产线库存:{data}')
16             condition.notify_all()
17             condition.release()
18
19
20 def consumer():
21     while True:
22         print(f'{threading.current_thread().name}:准备获取锁 ...')
23         condition.acquire()
24         print(f'{threading.current_thread().name}:已经获取锁 ...')
25         if not data :
26             print(f'{threading.current_thread().name}:waiting ...')
27             condition.wait()
28             print(f'{threading.current_thread().name}:wait 被唤醒')
29         else:
30             print(f'消费者取走{data.pop()} --- 目前数据:{data}')
31             condition.notify_all()
32             condition.release()
33
34 if __name__ == '__main__':
35     data = []
36
37     condition = threading.Condition()
38
```

```
39     t_producer = threading.Thread(target=producer,name="生产线程")
40     t_producer.start()
41
42
43     t_consumer = threading.Thread(target=consumer,name="消费线程")
44     t_consumer.start()
45
46
47
```

queue完成线程通讯

```
1 import random
2 import threading
3 import queue
4
5 def producer():
6     while True:
7         if not q.full():
8             data = random.randint(1,100)
9             q.put(data)
10            print(f'生产存入{data} 队列长度{q.qsize()}')
11
12
13 def consumer():
14     while True:
15         if not q.empty():
16             data = q.get()
17             print(f'消费取出{data} 队列长度{q.qsize()}')
18
19 if __name__ == '__main__':
20     data = []
21
22     q=queue.Queue(5)
23
24     t_producer = threading.Thread(target=producer,name="生产线程")
25     t_producer.start()
26
27
28     t_consumer = threading.Thread(target=consumer,name="消费线程")
29     t_consumer.start()
30
```

信号量锁机制完成线程控制

```
1 import threading
2 import time
3
4 semaphore = threading.BoundedSemaphore(3)
5
6 def func():
7     if semaphore.acquire():
8         print(f'{threading.current_thread().name} is working ...')
9         time.sleep(1)
10        print(f'{threading.current_thread().name} 完成任务...')
11        semaphore.release()
12
13
14 if __name__ == '__main__':
15     #模拟程序创建大量子线程
16     for i in range(10):
17         t = threading.Thread(target=func)
18         t.start()
```

Barrier控制线程执步调

```

1 import threading
2 import time
3 import random
4
5 def player():
6     time.sleep(random.randint(1,5))
7     print(f'{threading.current_thread().name} 抵达栅栏的时间为[1] {time.ctime()}')
8     b.wait()
9     #希望所有子线程全部抵达后才继续向后执行
10    print(f'{threading.current_thread().name} 继续向后执行[1]')
11    time.sleep(random.randint(1, 5))
12    print(f'{threading.current_thread().name} 抵达栅栏的时间为[2] {time.ctime()}')
13    b.wait()
14    print(f'{threading.current_thread().name} 继续向后执行[2] {time.ctime()}')
15    pass
16
17 if __name__ == '__main__':
18     print("比赛开始")
19
20     #设置需要拦住4个线程
21     b=threading.Barrier(4)
22     threads = []
23     for i in range(4):
24         t = threading.Thread(target=player,name=f"线程{i+1}")
25         threads.append(t)
26         t.start()
27
28     for th in threads:
29         th.join()
30
31     print("比赛结束....")

```

Event事件

```

1  import threading
2  import time
3  import random
4
5
6  def waiter():
7      for i in range(1,4):
8          print(f'{i} 等待flag被设定.')
9          #需求：等待线程如果需要向下执行，需要设定线程，修改flag标记
10         event.wait()
11         print(f'{i}等待完成时间: {time.ctime()}')
12         #需要重置/清空事件的等待状态
13         event.clear()
14
15
16  def setter():
17      for i in range(1, 4):
18          time.sleep(random.randint(1,3))
19          event.set()
20          print(f'{i}设定完成')
21
22  if __name__ == '__main__':
23      #创建一个event事件对象
24      event = threading.Event()
25      print('工作开始...')
26      w = threading.Thread(target=waiter)
27      s = threading.Thread(target=setter)
28      w.start()
29      s.start()
30      w.join()
31      s.join()
32      print("工作结束")

```

协程

yield实现协程运行

```

1
2
3 def task1():
4     for i in range(3):
5         print(f"-----task1 i = {i}-----")
6         yield
7
8 def task2():
9     for i in range(3):
10        print(f"-----task2 i = {i}-----")
11        yield
12
13
14 if __name__ == '__main__':
15     #yield让函数调用返回生成器对象，将当前任务函数挂起
16     g1=task1()
17     g2=task2()
18     #yield配合next实现调用
19     next(g1)
20     next(g2)
21
22     next(g1)
23     next(g2)
24
25     next(g1)
26     next(g2)
27
28     #调用次数超过任务代码执行次数会报错
29     next(g1)
30     next(g2)

```

生成器的3种取值方式

```
1 def fibonaccci(n):
2     a,b=1,1
3     current_n = 0
4     while current_n < n:
5         yield a
6         a,b=b,b+a
7         current_n+=1
8
9     #退出循环
10    return 'Hello World !'
11
12
13
14 if __name__ == '__main__':
15     v1 = fibonaccci(5)
16     print(next(v1))
17     print(next(v1))
18     print(next(v1))
19     print(next(v1))
20     print(next(v1))
21     # 报错
22     # print(next(v1))
23
24     print('-'*50)
25
26     v2 = fibonaccci(5)
27     while True:
28         try:
29             g = next(v2)
30             print(g)
31         except StopIteration as e:
32             print(e)
33             break
34
35     v3 = fibonaccci(5)
36     for item in v3:
37         print(item)
```

send向生成器传递值

```
1
2
3 def fibonaccci(n):
4     a,b=1,1
5     current_n = 0
6     while current_n < n:
7         r=yield a
8         print(f'r={r}')
9         a,b=b,b+a
10        current_n+=1
11
12    #退出循环
13    return 'Hello World !'
14
15
16 if __name__ == '__main__':
17     v = fibonaccci(5)
18     #print(next(v))
19
20    #第一次激活生成器，使用send方法传递的参数必须为None
21    print(v.send(None))
22    #或者通过next激活
23    #print(next(v))
24    print(v.send("hi"))
25    print(v.send("hello"))
26
```

yeild_from实现协程与执行特性

```

1  def generator_func():
2      total = 0
3      while True:
4          x = yield
5          print(f"add x:{x}")
6          # 0,0.0 '',[],{} 都为False
7          if not x :
8              break
9          total+=x
10     return total
11
12 def add_generator():
13     while True:
14         total = yield from generator_func()
15         print(f'加数的总和为:{total}')
16
17 if __name__ == '__main__':
18     # gen = generator_func()
19     # next(gen)
20     # gen.send(1)
21     # gen.send(2)
22     # gen.send(3)
23     # gen.send(0)
24
25     gen = add_generator()
26     next(gen)
27     gen.send(1)
28     gen.send(2)
29     gen.send(3)
30     gen.send(0)
31     gen.send(4)
32     gen.send(5)
33     gen.send(0)

```

await

调用协程任务

```

1 import asyncio
2
3
4 async def say_hi():
5     print("Hello")
6     #事件循环中等待，CPU可以执行其他任务
7     await asyncio.sleep(2)
8     print("World")
9
10 if __name__ == '__main__':
11     asyncio.run(say_hi())

```

协程并行任务(错误示范)

```

1 import asyncio
2 import time
3
4 async def say_delay(msg, sec):
5     await asyncio.sleep(sec)
6     print(msg)
7
8
9 async def main():
10     print("main() begin at {}".format(time.strftime("%H:%M:%S", time.localtime())))
11     #IO阻塞调用(这样挂起没有将任务放入协程事件循环中),3个任务串行执行，总耗时6秒
12     await say_delay("Hello", 1)
13     await say_delay("world", 2)
14     await say_delay("Welcome", 3)
15     print("main() end at {}".format(time.strftime("%H:%M:%S", time.localtime())))
16
17 if __name__ == '__main__':
18     asyncio.run(main())

```

协程并行任务(task)

```
1 import asyncio
2 import time
3
4 async def say_delay(msg, sec):
5     await asyncio.sleep(sec)
6     print(msg)
7
8
9 async def main():
10     print("main() begin at {}".format(time.strftime("%H:%M:%S", time.localtime())))
11
12
13     #3个任务并行执行，总耗时3秒
14     task1 = asyncio.create_task(say_delay("Hello", 1))
15     task2 = asyncio.create_task(say_delay("world", 2))
16     task3 = asyncio.create_task(say_delay("Welcome", 3))
17
18     await task1
19     await task2
20     await task3
21
22     print("main() end at {}".format(time.strftime("%H:%M:%S", time.localtime())))
23
24 if __name__ == '__main__':
25     asyncio.run(main())
```

协程并行任务(gather)

```
1 import asyncio
2 import time
3
4 async def say_delay(msg, sec):
5     await asyncio.sleep(sec)
6     print(msg)
7
8
9 async def main():
10     print("main() begin at {}".format(time.strftime("%H:%M:%S", time.localtime())))
11     #3个任务并行执行，总耗时3秒
12     task1 = asyncio.create_task(say_delay("Hello", 1))
13     task2 = asyncio.create_task(say_delay("world", 2))
14     task3 = asyncio.create_task(say_delay("Welcome", 3))
15     await asyncio.gather(task1, task2, task3)
16     print("main() end at {}".format(time.strftime("%H:%M:%S", time.localtime())))
17
18 if __name__ == '__main__':
19     asyncio.run(main())
```